Hardware-Aware KV Cache Quantization for Mobile NPUs

Jeongwoo Han
Department of Digital Media
and Communications Engineering
Sungkyunkwan University
jwhot.han@skku.edu

Dongkun Shin*
Department of Electrical
and Computer Engineering
Sungkyunkwan University
dongkun@skku.edu

***Abstract*—Key–value (KV) cache quantization reduces storage, but encoding and decoding overhead can limit its benefits on mobile neural processing units. We present a hardware-aware K8 codec that combines Piecewise-Uniform K8 (PU-K8) reconstruction levels with arithmetic indexing and lookup-free Hexagon Vector eXtensions (HVX) decoding. This design targets the eight-bit key path of a mixed-precision cache, retaining Lloyd–Max for K4/V2. We compare it with Lloyd–Max K8 under matched storage budgets and Odd and Boundary layer placements. We evaluate Qwen3-1.7B (W4A16) on a Galaxy S26 Ultra. Across ten warmed runs per configuration at a context capacity of 1024 tokens, PU-K8 reduces mean time to first token (TTFT) by 31.12–32.62% and increases mean autoregressive decoding throughput by 13.61–14.41%. On four reused WikiText-2 reference windows, perplexity (PPL) relative to Lloyd–Max K8 is 1.81% higher under Odd placement and 0.45% lower under Boundary placement. Needle-in-a-Haystack retrieval at 1K–8K capacities yields 93.5% answer-code extraction accuracy for Boundary PU-K8, versus 95.5% for Boundary Lloyd–Max K8 and 81.0% for TurboQuant K4/V4. These results show that codec implementation affects inference efficiency at a fixed KV cache budget, with observed quality differences depending on placement and evaluation set.**

***Index Terms*—KV cache, quantization, mobile NPU, piecewise-uniform quantization, Hexagon HVX**

I. Introduction

Key–value (KV) cache quantization reduces storage and data movement, but encoding and decoding overhead can limit its benefits for mobile inference. KIVI motivates asymmetric key/value quantization [1], while TurboQuant explores rotation-based scalar quantization [2]. Practical deployment therefore requires evaluating codec costs alongside quantization error.

Our work targets the K8 codec in a TurboQuant implementation in the Qualcomm AI Hub Models pipeline. Its Lloyd–Max encoder uses comparison trees and reconstruction-level selection subgraphs, while lookup-table (LUT) decoding requires table lookup and layout conversion. These costs motivate designing the quantization levels and their execution paths together.

Building on piecewise quantization approaches such as PWLQ [3] and PTQ4ViT [4], our contributions are twofold. *K8 codec design and implementation:* Piecewise-Uniform K8 (PU-K8) combines two-region levels with arithmetic indexing and lookup-free HVX decoding in a rotated mixed-precision cache, preserving the existing byte format and explicit FP16 rounding. K4/V2 retain Lloyd–Max. *On-device evaluation:* matched comparisons with Lloyd–Max K8 and a separate single-spacing uniform baseline quantify execution cost and quality trade-offs under fixed storage budgets and layer masks. Figure 1 summarizes the design.

*Corresponding author: Dongkun Shin.

II. Background and Motivation

A. Key and Value Quantization Errors

Scaled dot-product attention is [5]

$$O = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \tag{1}$$

where Q, K, and V are queries, keys, and values, and d is the key dimension; causal masking is omitted. Key quantization changes the dot products and thus the normalized exponential softmax weights. In contrast, value errors enter the weighted sum linearly for fixed Q and K. This motivates asymmetric K/V precision, without implying that key errors always dominate.

B. Layer-Wise Quantization Sensitivity

KV quantization sensitivity varies across layers [6], motivating higher precision in selected layers under a fixed cache budget. We mix K8 and K4 to maintain average K6 precision, with V2 throughout. Odd and Boundary are tested placements, not presumed optima; comparisons use matched masks.

C. Codec Execution Costs

In the inherited K8 implementation, Lloyd–Max encoding uses boundary comparisons and reconstruction-level selection, while LUT decoding loads the selected levels. Piecewise-uniform levels enable arithmetic indexing and native HVX reconstruction. Both K8 codecs store one byte per coordinate; K4/V2 retain their existing packed paths. Thus, the quantization-level structure and its execution path affect codec cost even at the same storage precision.

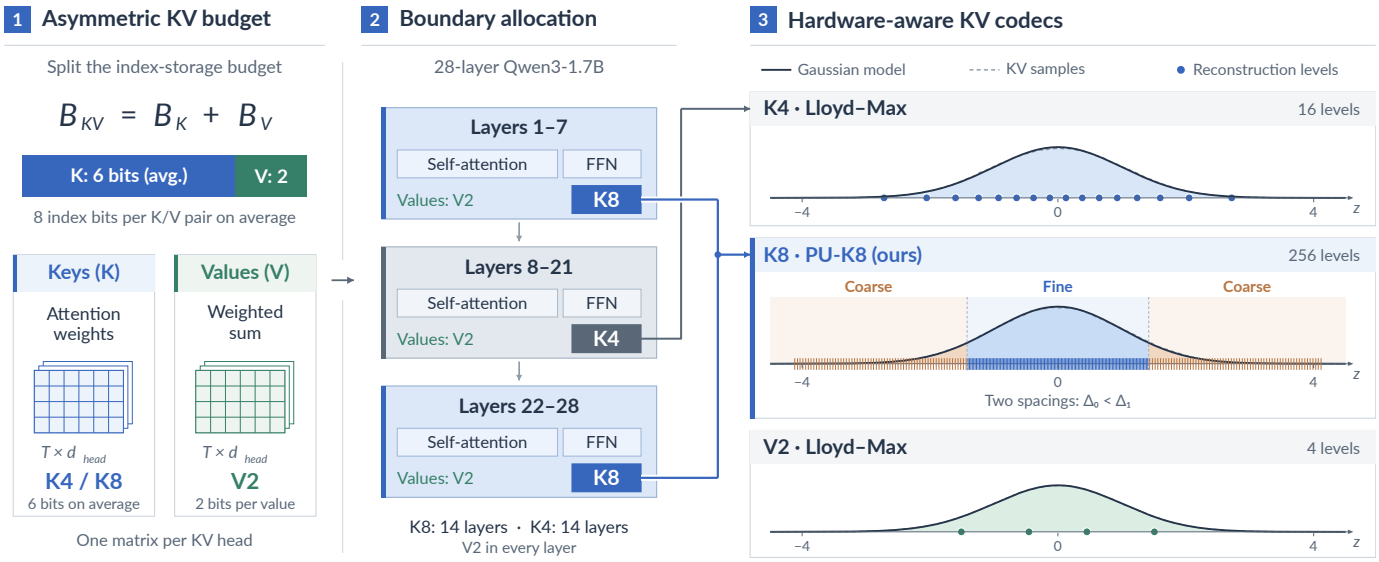


Fig. 1. Hardware-aware K8 codec in a mixed-precision KV cache with Boundary placement. Connectors map each layer group's K precision to its fixed codec; K4/V2 retain Lloyd–Max. The right panel shows the Gaussian design model and all frozen FP16 reconstruction levels (16/256/4 for K4/K8/V2). Gray traces use captured Qwen3-1.7B KV samples with CPU-emulated normalization and rotation, plotted as $z = \sqrt{d}\,y$ with $d = 128$.

III. Hardware-Aware K8 Codec Design

A. Overview

The three panels in Fig. 1 connect the storage budget to the deployed codecs. In the left panel, B_K and B_V denote key and value index-storage budgets, so $B_{KV} = B_K + B_V$ excludes FP16 scales. The cache dimensions are token count T, KV-head count n_{heads}, and head dimension d_{head}. Assigning K8 to half of the layers and K4 to the rest gives an average key precision of six bits; values use V2 throughout.

The center panel illustrates Boundary placement, with K8 assigned to the first and last seven of 28 layers. The drawing numbers layers from 1 to L; our experimental masks use zero-based indices. Odd placement is also evaluated under the same storage budget. These placements instantiate the precision allocation used to compare quantizer implementations.

The connectors map each layer group's allocated key precision to a fixed codec: Lloyd–Max for K4 and PU-K8 for K8; all values retain Lloyd–Max V2. The standardized axis $z = \sqrt{d}\,y$ rescales the normalized, rotated coordinate y used below. The plots overlay sample densities and the Gaussian design model, with the deployed FP16 reconstruction levels. The following subsections specify arithmetic indexing and native HVX decoding.

B. Quantization Levels and Arithmetic Indexing

The proposed key cache uses Lloyd–Max K4 and PU-K8. For bit width p, the positive reconstruction levels are

$$\ell_p^+(j) = \begin{cases} \mathcal{L}_4^+[j], & p = 4, \\ a_r + k\Delta_r, & p = 8. \end{cases} \tag{2}$$

For K4, $\mathcal{L}_4^+$ contains eight positive Lloyd–Max levels indexed by $j \in \{0, \ldots, 7\}$; the threshold-based encoder is retained. For K8, $j = 64r + k$, with region $r \in \{0, 1\}$, local index $k \in \{0, \ldots, 63\}$, offset a_r, and spacing Δ_r. Negative levels follow by sign reflection. The K8 expression describes the grid before rounding; its multiplication and addition are separately rounded to FP16.

The four coefficients are fitted offline to minimize scalar mean squared error (MSE) under the Gaussian approximation $\mathcal{N}(0, 1/d)$, without KV samples or PPL-based tuning.

Arithmetic indexing. For a normalized, rotated coordinate y, let $m = |y|$. The boundary τ is the midpoint between the FP16-rounded levels at $(r, k) = (0, 63)$ and $(1, 0)$. Select the region as

$$r = \begin{cases} 0, & m \le \tau, \\ 1, & m > \tau. \end{cases} \tag{3}$$

The local index is

$$k = \operatorname{clip}\left(\left\lfloor (m - a_r)\frac{1}{\Delta_r} + 0.5 \right\rfloor, 0, 63 \right) \tag{4}$$

The logical floor rule rounds half-step ties upward. Execution uses a stored reciprocal with intermediate FP16 rounding, which can shift decisions near index boundaries.

Byte encoding. The magnitude index j and the sign of y are stored in one UINT8 code:

$$b = \begin{cases} 128 + j, & y \ge 0, \\ 127 - j, & y < 0. \end{cases} \tag{5}$$

For negative values, the magnitude index is reflected to preserve compatibility with the existing ascending LUT layout. The decoder recovers $j = b - 128$ for $b \ge 128$ and $j = 127 - b$ otherwise.

C. HVX Arithmetic Decoder

The native HVX decoder recovers r and k from each stored byte and reconstructs $a_r + k\Delta_r$ using separate FP16 multiply and add operations. It then applies the sign and the effective FP16 scale used for norm correction, producing FP16 tiles for the existing attention path. Vectorized decoding avoids level-table loads and per-coordinate scalar loops. The coefficient payload is four FP16 values (8 bytes), replacing a 256-entry FP16 codebook (512 bytes).

IV. Experimental Methodology

Experiments use Qwen3-1.7B [7] (W4A16) on a Galaxy S26 Ultra (SM8850), with QAIRT 2.48.0.260626 and the HTP backend in burst mode. Inference uses a single sequence; performance and PPL evaluations use a context capacity of 1024. PU-K8 and Lloyd–Max K8 are compared under matched masks in Odd and Boundary placement. Weights, activation encodings, rotations, norm correction, and K4/V2 paths remain fixed.

Performance uses 897 prompt tokens and 128 generated tokens. Each invocation includes a full warm-up, a cache reset, and a measured session. TTFT excludes model loading; decode throughput is 127 divided by the total duration of the post-prefill autoregressive steps. The Lloyd–Max comparison uses ten measured sessions per configuration in counterbalanced order, with a ten-second pause between invocations. We report means and sample standard deviations, with improvements computed from group means. The same session logs separate host-observed QNN graph-call time from the remaining runner time.

Reference-window PPL is evaluated by teacher forcing on four reused WikiText-2 [8] windows of 1024 tokens, using the unchanged performance binaries. Each window is scored once per configuration, totaling 4092 target tokens per configuration. Aggregate perplexity is the exponential of the token-averaged negative log-likelihood (NLL) across all four windows. The Uniform K8 comparison additionally evaluates PPL on a separate four-document set under Odd placement.

Needle-in-a-Haystack (NIAH) retrieval uses ten fixed WikiText-2 backgrounds containing a six-digit target code, five insertion depths (0, 25, 50, 75, and 100%), and context capacities of 1K, 2K, 4K, and 8K tokens (1K = 1024). Each configuration has 200 prompts, each decoded once using non-thinking greedy generation. Prompts fit within capacity minus 64 tokens, reserving up to 64 output tokens. Accuracy requires exactly one standalone six-digit code matching the target. We compare FP16 KV/attention, TurboQuant K4/V4, and the two Boundary K8/K4 with V2 configurations.

V. Results and Analysis

A. End-to-End Performance and Quality

We compare PU-K8 with Lloyd–Max K8, which has lower Gaussian scalar reconstruction MSE. This comparison evaluates complete implementations.

Under Boundary placement, Fig. 2 shows that PU-K8 reduces mean TTFT by 31.12% and increases prefill and decode throughput by 45.22% and 14.41%, respectively. Separate Odd-placement runs show a 32.62% TTFT reduction and 48.49% and 13.61% higher prefill and decode throughput, respectively. The speed advantage is observed under both tested masks on this device.

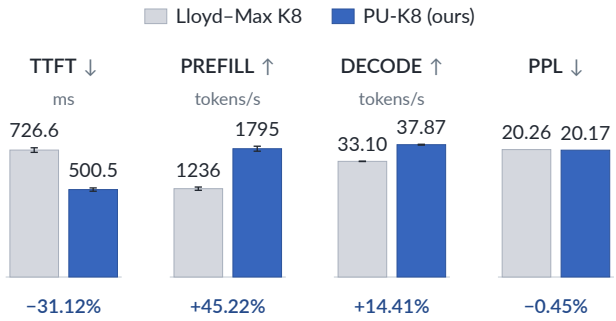


Fig. 2. Boundary-placement performance and reference-window PPL. Timing: mean ± sample SD over 10 runs per configuration, with 897 prompt and 128 generated tokens. Percentages show changes from Lloyd–Max K8.

In the same Boundary sessions, mean total QNN graph-call time falls from 697.47 to 475.11 ms during prefill and from 28.43 to 24.63 ms per generated token. Total token latency falls from 30.21 to 26.41 ms, while the remaining time stays near 1.78 ms/token. Thus, the savings are concentrated within graph calls, which include execution and scheduling/wait time.

TABLE I
Reference-window PPL reduction from Odd to Boundary placement.

K8 codec	Odd PPL ↓	Boundary PPL ↓	Reduction (%)
Lloyd–Max	23.19	**20.26**	12.65
PU-K8	23.61	**20.17**	14.58

On the four reused reference windows (Table I), Boundary lowers PPL by 12.65% for Lloyd–Max and 14.58% for PU-K8 relative to Odd. Relative to Lloyd–Max, PU-K8's PPL is 1.81% higher under Odd and 0.45% lower under Boundary, reversing the aggregate ranking. Lloyd–Max wins three Odd windows; each codec wins two Boundary windows.

B. Needle-in-a-Haystack Retrieval

Figure 3 reports answer-code extraction accuracy over 200 prompts per configuration. Boundary PU-K8 scores 93.5%, 2.0 percentage points lower than Boundary Lloyd–Max K8 (95.5%); TurboQuant K4/V4 scores 81.0%.

C. Comparison with Uniform Quantization

The uniform baseline uses a symmetric 256-level grid with one spacing fitted under the same Gaussian model, without KV- or PPL-based tuning, and frozen to FP16. Both variants use arithmetic HVX decoders and the same Odd placement, allowing a comparison of single- and two-region quantization under matched deployment settings.

In Fig. 4, PU-K8 reduces PPL by 11.28% on the separate four-document set. On the reference windows, PU-K8 reduces

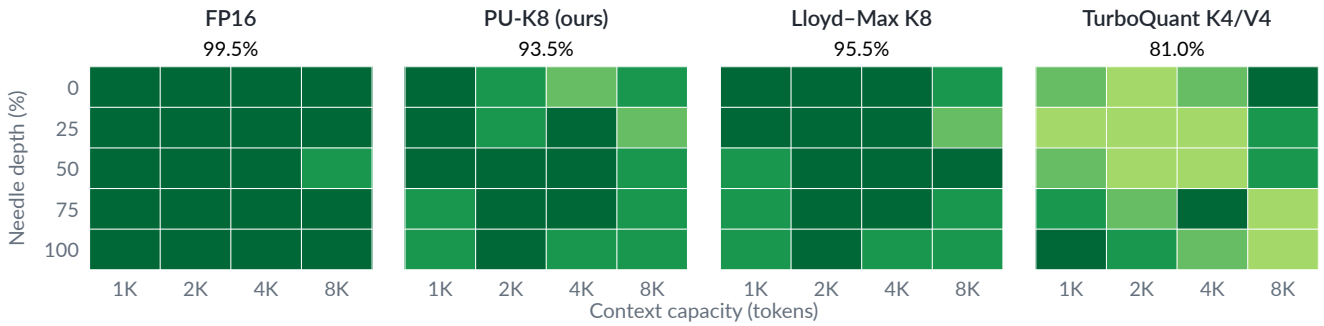


Fig. 3. NIAH answer-code extraction accuracy of Qwen3-1.7B (10 cases per cell). Both K8 codecs use Boundary placement; the three quantized configurations share the same KV cache storage budget at each context capacity.

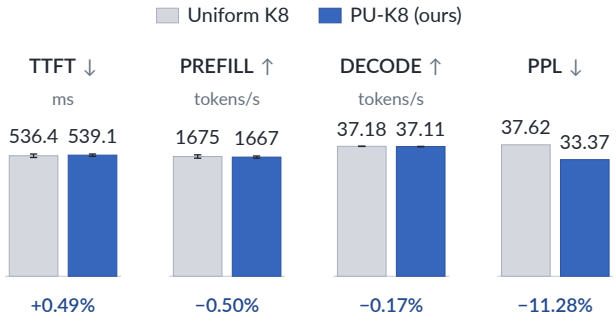


Fig. 4. Odd-placement comparison with Uniform K8. PPL uses a separate four-document set. Timing: mean ± sample SD over three runs per configuration, with 897 prompt and 128 generated tokens. Percentages show changes from Uniform K8.

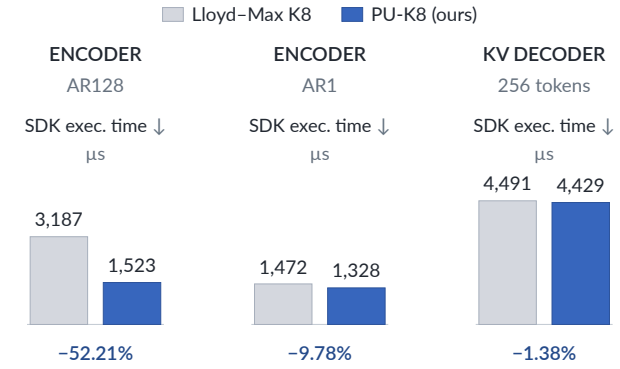


Fig. 5. Single-probe SDK execution times for AR128/AR1 encoders and 256-token KV decoders. Percentages show changes from Lloyd–Max K8; probe times are not additive.

Uniform PPL by 0.76% (23.79 to 23.61), also under Odd placement. The benefit varies by evaluation set; absolute PPL values across sets are not directly comparable. In the separate set, NLL improves on three documents, but most of the net gain comes from one. Three-run timings differ little and have overlapping ranges, showing no clear speed advantage.

D. Encoder and Decoder Analysis

Autoregressive decoding includes new-KV encoding, cached-KV decoding, and attention. PU-K8 replaces repeated boundary comparisons and level selection with arithmetic indexing and reconstruction, reducing the encoder graph from 837 to 76 ONNX nodes. Single probes with matched inputs show 52.21% lower SDK execution time for AR128 (prefill) and 9.78% for AR1 (token generation), as shown in Fig. 5. The stronger AR128 encoder reduction is consistent with the larger prefill speedup.

For cached-KV decoding, arithmetic reconstruction avoids LUT loads. The 256-token probe shows only 1.38% lower SDK execution time. With identical input codes and scales, each decoder matches its own FP16 reference bit-exactly.

These isolated probes differ in shape and execution conditions from the full model and cannot be summed to explain its speedup. The session logs locate the savings within graph calls but do not separate encoder, KV-decoder, and attention contributions.

VI. Conclusion

KV cache efficiency depends on codec execution cost as well as storage precision. On the evaluated mobile NPU, PU-K8 reduces mean TTFT by 31–33% and increases decode throughput by about 14% versus Lloyd–Max K8. Observed quality differences vary with placement and evaluation set. Further evaluation requires larger independent datasets, additional models and devices, and broader long-context tasks.

References

[1] Z. Liu *et al.*, “KIVI: A tuning-free asymmetric 2bit quantization for KV cache,” arXiv:2402.02750, 2024.
[2] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, “TurboQuant: Online vector quantization with near-optimal distortion rate,” arXiv:2504.19874, 2025.
[3] J. Fang, A. Shafiee, H. Abdel-Aziz, D. Thorsley, G. Georgiadis, and J. Hassoun, “Post-training piecewise linear quantization for deep neural networks,” arXiv:2002.00104, 2020.
[4] Z. Yuan, C. Xue, Y. Chen, Q. Wu, and G. Sun, “PTQ4ViT: Post-training quantization for vision transformers with twin uniform quantization,” in *Proc. ECCV*, 2022.
[5] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
[6] X. Li *et al.*, “KVTuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization for efficient and nearly lossless LLM inference,” in *Proc. ICML*, PMLR, vol. 267, pp. 36451–36485, 2025.
[7] A. Yang *et al.*, “Qwen3 technical report,” arXiv:2505.09388, 2025.
[8] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” arXiv:1609.07843, 2016.